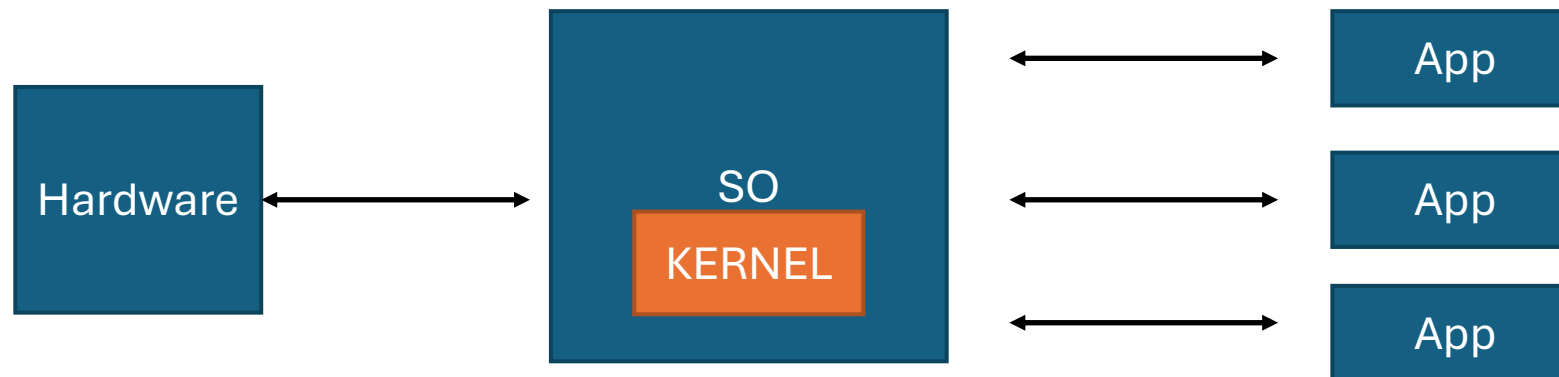


Kernel y Procesos

¿Qué es un Kernel?

- El Kernel, como lo hemos mencionado, es el corazón del Sistema Operativo. Es el encargado de la separación de aplicaciones y de mantener “Segura” la máquina.
- En general esto es un patrón de trabajo donde alguien interviene entre 2 partes para que ellas se mantengan aisladas (proxy)



Funciones

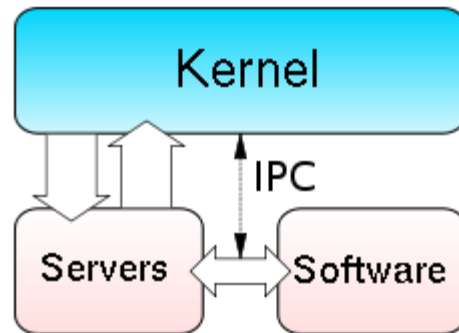
- Un Kernel se encarga de:
 - Garantizar la ejecución de los procesos: programas, entrada/salida
 - Separar los recursos usados por cada aplicación en cada momento
 - Establecer el estándar de comunicación de los programas
 - Gestión de memoria, procesador y periféricos
- Las funciones de comunicación y almacenamiento físico normalmente NO son parte de los núcleos. Pero su gestión se administra como un proceso más dentro del mismo. Es decir, residen en él directamente.

Tipos de Kernel (1)

- Monolítico: están directamente ligados al hardware y TODO el SO es el Núcleo (kernel).
- No es modular, y agregar una nueva funcionalidad puede requerir toda una recompilación.
- Ejemplos:
 - Linux
 - Unix
 - DOS
 - MacOS hasta el 8.6

Tipos de Kernel (2)

- Microkernel: es un Sistema operativo que tiene un núcleo que se encarga de comunicación y planificación de procesos: pero TODO lo demás (Entrada Salida, Memoria, Archivos) lo delega
- De esta forma: cuando un proceso necesita memoria se la pide a un MICROGESTOR/SERVIDIRO DE MEMORIA. Igual si es entrada/salida, archivos, etc.



La idea es tener SO más optimos para multitareas y que los recursos estuvieran todavía más aislados, también que los errores fueran más aislados.

Tienen el problema de la sincronización entre servidores/gestores es MUY COMPLEJA. También son más difíciles de programar. **LEAD**

Tipos de Kernel (2.1)

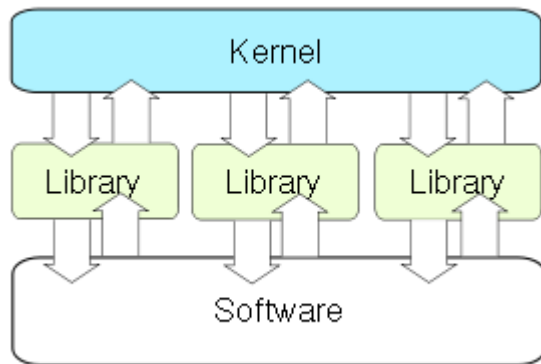
- Ejemplos de SO de Microkernel
 - AmigaOS
 - Minix
 - NexStep (lo consideran híbrido)
 - AIX

Tipos de Kernel (3)

- Núcleos Híbridos: son sistemas que mezclan la filosofía Monolítica con MicroKernel.
- Una parte queda en el Kernel y otra la exponen para que cada aplicación/proceso la gestione individualmente.
- Es la tendencia más moderna. Asumida por Windows desde Windows NT y Mac OS X.

Tipos de Kernel (y 4)

- Exonúcleo: son sistemas hasta ahora solo usados en INVESTIGACIÓN. La idea es que TODO sea proporcionado a las aplicaciones por gestores específicos donde el KERNEL sería solamente las operaciones más básicas (sencillamente el bios y las abstracciones más básicas de hardware).



Si pudiéramos compararlo con algo más concreto, sería como decir una arquitectura de Microservicios.

¿Cómo se carga el sistema operativo?

- **Encendido:** el procesador comienza ejecutando el firmware almacenado en la placa base.
- **BIOS/UEFI:** realiza el POST, detecta componentes y deja el hardware listo para arrancar.
- **Dispositivo de arranque:** busca un SSD/HDD, USB o red según el orden configurado.
- **Cargador de arranque:** el firmware lo invoca y le transfiere el control.
- **Kernel:** el cargador lo coloca en la RAM; después inicializa memoria, controladores y servicios.

Relación clave: el firmware prepara y entrega; el sistema operativo toma el control cuando empieza a ejecutarse el kernel.

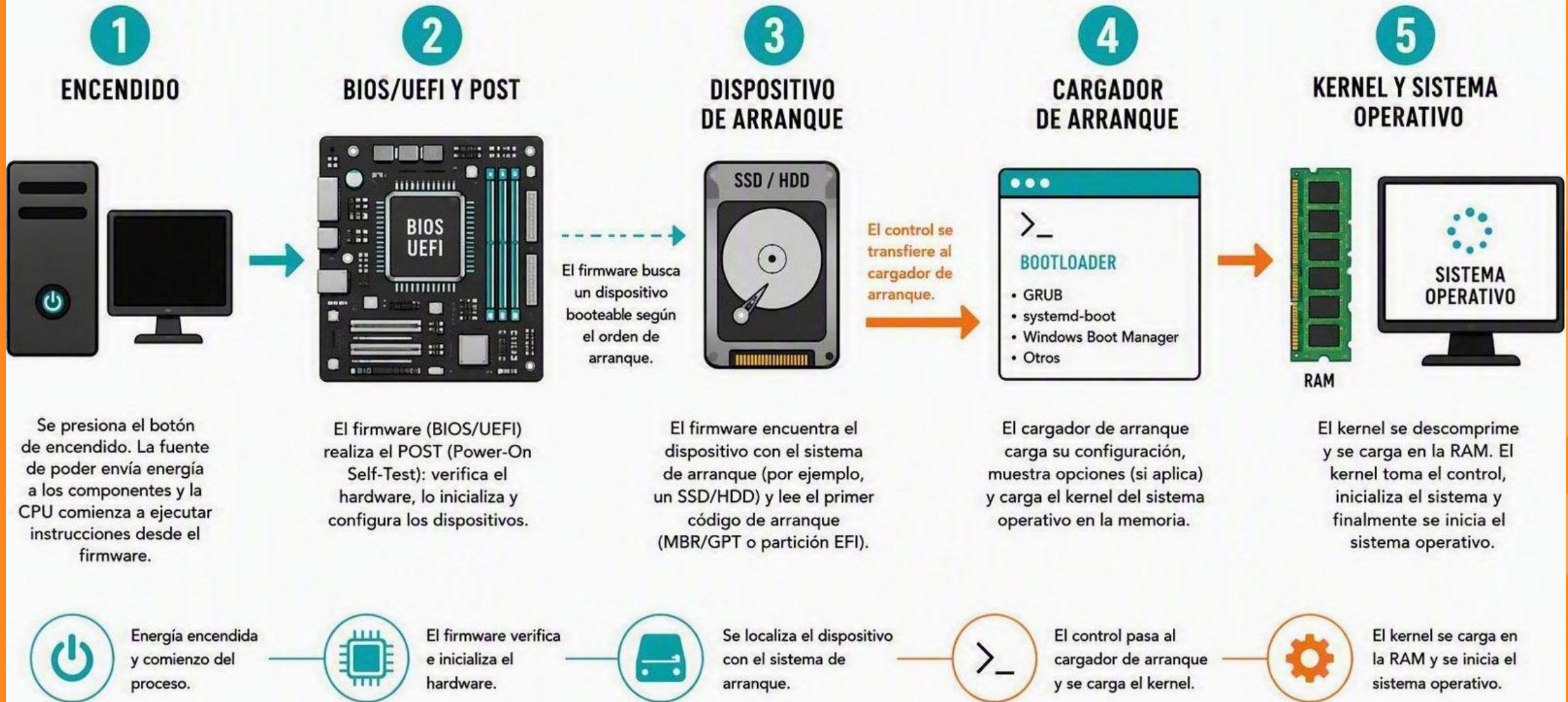
Paso a paso...



**El firmware no es el sistema operativo: su función termina al entregar el control.
Desde el kernel, el sistema operativo administra la máquina.**

ARRANQUE DE LA COMPUTADORA

RECAPITULACIÓN VISUAL



Ya con el S.O. cargado...

- El Kernel tiene acceso a TODA la máquina.
 - Se ejecuta directamente en el procesador, SIN LIMITACIONES e interactúa con él a conveniencia.
 - Puede hacer CUALQUIER COSA.
- Un Proceso es UNA ABSTRACCIÓN INTERNA PARA la ejecución de un programa.
 - Un PROGRAMA es una unidad INACTIVA. Un PROCESO es una entidad active.
 - Tiene RESTRICCIONES: no puede ver libremente todo.
 - Un proceso NECESITA permisos del kernel para acceder a un espacio de memoria, leer o escribir del disco.
 - TASK o Tarea se puede considerar sinónimo de PROCESO.

Los Procesos y el Kernel (y 2)

- ¿Cómo pueden correr al mismo tiempo procesos restringidos y un kernel en un mismo procesador, la misma memoria y mismos recursos?
- Piense en el procesador como en alguien con 2 personalidades
 - Cuando está con “el kernel” lo puede todo.
 - Cuando está con un proceso: pues no.

Procesos y el Kernel

Ejecutable
(compilado)

Programa

SO

Memoria Física



El SO SEPARA un espacio en memoria para guardar los datos que el programa crea dinámicamente en su ejecución y las llamadas a procedimientos

(recuerdan compiladores? Las PILA DE LLAMADOS)

El SO se carga en memoria al inicio.

Cuando se desea ejecutar un programa, el Sistema operativo COPIA las instrucciones y datos a la memoria física.

Se puede ejecutar MÚLTIPLES veces un mismo programa al mismo tiempo. Cada vez, se reservará espacios DIFERENTES EN MEMORIA.

Cada “ejecución” en memoria es UN PROCESO.

Un Proceso es UNA INSTANCIA DE EJECUCIÓN de un Programa compilado de la misma manera que un Objeto es una

Process Control Block (1)

- El Sistema operativo controla los procesos de la computadora usando una estructura de datos llamada PROCESS CONTROL BLOCK.
- Aquí se guarda toda la información de cada proceso:
 - Donde está en la memoria,
 - Donde está el archivo Fuente
 - Cuál usuario ordenó su ejecución
 - Nivel de Privilegios

Contenido de un bloque PCB

Id del Proceso

Status

Estado:

- Instrucción actual
(process status word)
- Contenido de los registros
- Memoria Principal
- Recursos
- Prioridad

Contabilidad

Process Control Block (2)

- Id del Proceso: es un descriptor proporcionado por el *Job Scheduler*
- Status: cómo está el proceso en este momento: HOLD, READY, RUNNING o WAITING.

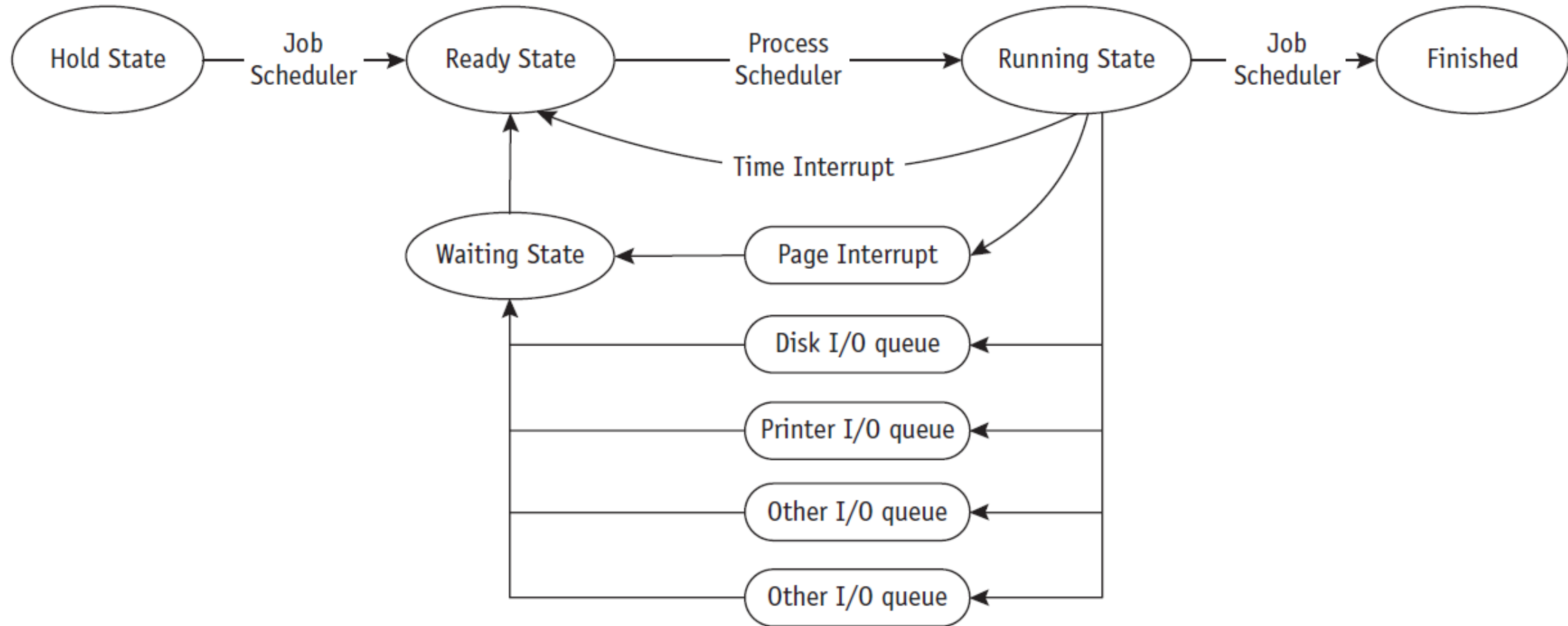
Process Control Block (y 3)

- Estado: la información de cómo estaba el proceso en el ultimo momento de ejecución
 - Instrucción Actual (Process Status Word) es la instrucción en el registro de instrucción antes de ponerse en HOLD, READY o WAITING. Si el proceso está corriendo, no hay nada
 - Contenidos de Registros: los contenidos de los registros de la CPU antes de interrumpir la ejecución
 - Memoria Principal: información pertinente al proceso: dirección de su espacio en memoria, y si usa Memoria Virtual, cual es el mapeo de la misma.
 - Recursos: todos los recursos asignados: cada uno identificado y detallado en cuanto a direcciones de memoria o sectores de disco.
 - Prioridad: usada por el Sistema para determinar CUANDO vuelve a ejecutarse.

Process Control Block

- Contabilidad: es la información de uso y rendimiento.
 - Cuanto tiempo CPU se ha usado desde su ejecución
 - Cuanto tiempo y espacio consume en memoria (y consumió en su ejecución)
 - Uso de memoria virtual
 - Cantidad y tipo de operaciones de E/S
 - Tiempo usado en E/S

Estados de Un proceso a ALTO NIVEL



Pero.... ¿Qué o quién es el Job Scheduler?

- El Job Scheduler es parte del PROCESSOR MANAGER.
 - El Processor Manager es quién lleva control de los procesos
 - Vigila si un proceso se está ejecutando o esperando operaciones de entrada salida.
 - Tiene 2 partes: el JOB SCHEDULER y el PROCESS SCHEDULER, ambas funciones son especialmente críticas hoy día donde hay procesadores con multiples cores (CPUs)
- El proceso de asignar tiempo de procesador a cada PROCESO se llama MULTIPROGRAMACIÓN.

Job Scheduling versus Process Scheduling (1)

- Job Scheduling se refiere a INICIAR o NO un proceso.
- Process Scheduling se refiere a QUE PARTE DEL PROCESO vamos a ejecutar.
 - Supongamos que usted DECIDE empezar a armar un lego. En ese momento el Job Scheduling le dió luz verde al JOB y empieza el PROCESO (la Tarea).
 - Naturalmente, el PROCESS SCHEDULING será abrir la caja, sacar las instrucciones e ir al Paso 1, Paso 2... Paso N... El paso a paso es el Process Scheduling.
 - Si lo llaman por telefono para poner a hervir los frijoles, eso es como una INTERRUPCIÓN por lo que usted PARA el PROCESO y el JOB SCHEDULING lo lleva a IR A PONER A HERVIR LOS FRIJOLES.
 - Usted MARCA donde quedó y que estaba haciendo, va y pone a hervir los frijoles (ejecuta la tarea o proceso)
 - Cuando termina, Vuelve a ocuparse del otro proceso: el Lego, retomando de donde quedó exactamente.

Job Scheduling versus Process Scheduling (y 2)

- El Job Scheduler tiene que tener mucho cuidado de no atascar procesos:
 - Por ejemplo saturar de E/S o de exceso de calculos porque eso haría que muchos procesos se trabaran.
 - Siempre tratará de mantener la mayor cantidad de dispositivos ocupados de forma equilibrada.

El Process Scheduler

- Como el Job Scheduler se encarga de dar paso a ejecución o no a los procesos, lo realmente interesante pasa con el Process Scheduler (PS).
- Este componente es de BAJO NIVEL. Y es el que asiga al CPU los procesos marcados como READY por el JOB SCHEDULER.
- El PS trata de aislar el Código de los programas por “bloques” de entrada salida y bloques de operación

El Process Scheduler

- Los bloques de E/S tiene normalmente pocos ciclos de CPU y largos ciclos de E/S
- Los bloques “de operación” tienen ciclos largos de operaciones de CPU y poco o nada E/S
- También en ocasiones, si un proceso está tomando mucho tiempo, puede “priorizar” otros para terminar más rápido y así dedicarle más tiempo luego al proceso

Entonces: ¿cómo se ejecuta un proceso?



- Cuando se ordena ejecutar un programa, es puesto en HOLD: y se coloca en una COLA DE EJECUCIÓN administrada por el JOB SCHEDULER. Se crea un PCB
- Cuando el JOB SCHEDULER lo saca de la cola, pasa a READY: que significa que puede ejecutarse pero espera por espacio en un CPU.
- RUNNING, obviamente significa que el proceso está en el CPU. El paso de READY a RUNNING lo hace el Process Scheduler. Un Proceso puede Volver de RUNNING a READY en ciertos momentos.
- WAITING significa que el programa no puede continuar hasta que UN RECURSO esté disponible o termine una operación de I/O. La transición de RUNNING a WAITING la hace el Process Scheduler y se da cuando se necesita LEER/ESCRIBIR de E/S o interactuar con datos de memoria
- En WAITING un proceso puede pasar a READY por el Process Scheduler cuando hay una interrupción o la E/S es satisfecha.
- FINISHED, pues representa el fin. La transición de RUNNING a FINISHED se puede dar
 - Por el PS cuando ya no hay mas que ejecutar
 - Por el Job Scheduler cuando hay un error o comando de terminación directa.

Políticas de “Process Scheduling”

- La programación de un PS puede hacerse con varios criterios:
 - Maximizar productividad (Throughput)
 - Minimizar tiempo de respuesta: priorizar los bloques de E/S
 - Maximizar rotación de procesos en el CPU
 - Minimizar tiempos de espera: sacar los procesos de READY lo antes possible
 - Maximizar la eficiencia del CPU manteniéndolo tan ocupado como sea possible
 - Todo mundo mismo tiempo de CPU
- NO hay política perfecta. Todas tienen pros y contra.

Algoritmos de Process Scheduling

- “First come, first served”
 - Significa que se da prioridad según llegan, más recursos a los que llegan primero. Filosofía FIFO.
- “Shortest next job”
 - Se estima cuando tiempo CPU va a usar un proceso, y se le da prioridad a los más cortos.
- Prioridad
 - Se da niveles de importancia (recuerden los PCB) a los procesos y cuanto mayor prioridad, más tiempo de CPU
- Próximo a terminar:
 - Se da prioridad al Proceso que está más cerca de terminar.
- Round-Robin:
 - A todos, misma prioridad en ciclo

Tiempos muertos

- Hay que recordar también que es importante que los PROCESOS (programas, al ejecutarse) reaccionen pronto para el usuario
- Y cada vez que el CPU hace “switch” de un proceso a otro hay un tiempo muerto.
- Es decir, hay que considerar que el costo de las transiciones NO es gratuito.